

Computer Architecture

Chapter 1(Part 2): Fundamentals of Quantitative Design & Analysis

Chapter 1: Fundamentals of Computer Design (Part 2)

What is computer architecture?

Why study computer architecture?

Common principles

Performance

What is performance: latency, throughput

The performance equation

Measuring performance

Improving performance: parallelism, locality, Amdahl's law

Power

Cost

Reliability

What is Performance?

Two Metrics

Latency (or response time or execution time)

Throughput (or bandwidth)

What is Performance?**

Two Metrics

Latency (or response time or execution time)

Time from start to finish of a task

$$*Performance = \frac{1}{execution\ time}*$$

Throughput (or bandwidth)

What is Performance?**

Two Metrics

Latency (or response time or execution time)

Time from start to finish of a task

Throughput (or bandwidth)

Rate of task completion

= Rate of task initiation

= $1 / (\text{time between task completions})$

What is Performance?**

Two Metrics

Latency (or response time or execution time)

Time from start to finish of a task

Throughput (or bandwidth)

Rate of task completion

= Rate of task initiation

= $1 / (\text{time between task completions})$

Example: Automobile assembly line starts one car per hour and holds 20 cars

Latency = 20 hours

Throughput = one car per hour

Throughput $> 1/\text{Latency}$ due to overlap

Performance (Cont.)

Speedup Definition: X is n% faster than Y if

$$\frac{\text{Execution Time}_Y}{\text{Execution Time}_X} = 1 + \frac{n}{100}$$

Speedup Definition: X is n times faster than Y if

$$n = \frac{\text{Execution Time}_Y}{\text{Execution Time}_X} = \frac{\text{Performance}_X}{\text{Performance}_Y}$$

Example: X = 1 minute, Y = 2 minutes

X is 100% faster than Y or 2 times faster than Y

Key Performance Equation

$$CPU_{time} =$$

$$CPU \text{ clock cycles for a program} * \text{Clock cycle time or } \left(\frac{1}{\text{Clock rate}} \right)$$

$$CPU_{time} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{time}}{\text{cycle}}$$

Instructions per program (**IC**, path length or count)

ISA and compiler

Cycles per instruction (**CPI**)

ISA and organization (e.g., cache misses)

Time per cycle (**CT**, clock time, cycle time)

Organization and hardware

Measuring Performance (Throughput)

MIPS (Million Instruction Per Second),

$$MIPS = \text{Instructions per cycle (IPC)} * \text{Clock rate} = \frac{\text{Clock rate}}{CPI}$$

FLOPS (Floating Point Operation Per Second)

MFLOPS (Million Floating Point Operation Per Second)

GFLOPS (Giga Floating Point Operation Per Second)

TFLOPS (Tera Floating Point Operation Per Second)

MIPS or MFLOPS don't mean much

Measuring Performance

Benchmarks

Real programs

- Representative of real workload

- Only way to characterize performance

- SPEC89 → SPEC92 → SPEC95 → SPEC CPU2000 → CPU2006 → CPU2017

- SPECFS, SPECWeb, SPECjbb, SPECvirt_Sc2010, TPC

Kernels

- ``Representative" program fragments

- Often not representative of full applications

- EEMBC for embedded systems

Toy benchmarks and synthetic benchmarks

- Don't mean much

Improving Performance – Basic Principles

Parallelism

Locality

Focus on common case – Amdahl's law

Improving Performance – Basic Principles**

Parallelism

- Pipelining (review next)

- Multiple issue

- Multiprocessors

Locality

Focus on common case – Amdahl's law

Review – Pipelining

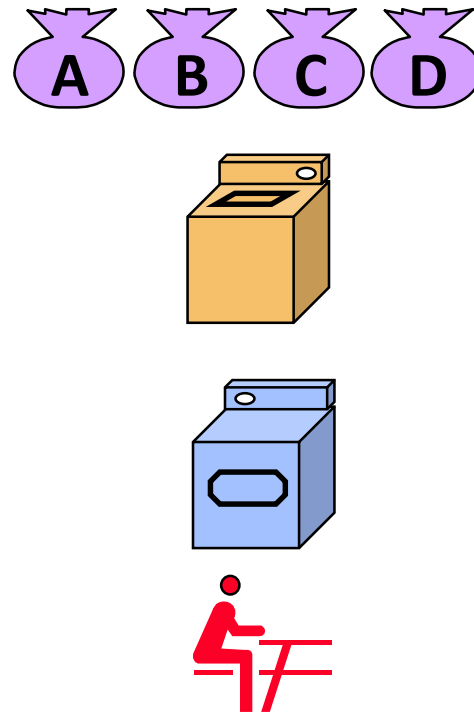
Laundry Example

Ann, Brian, Cathy, Dave
each have one load of clothes
to wash, dry, and fold

Washer takes 30 minutes

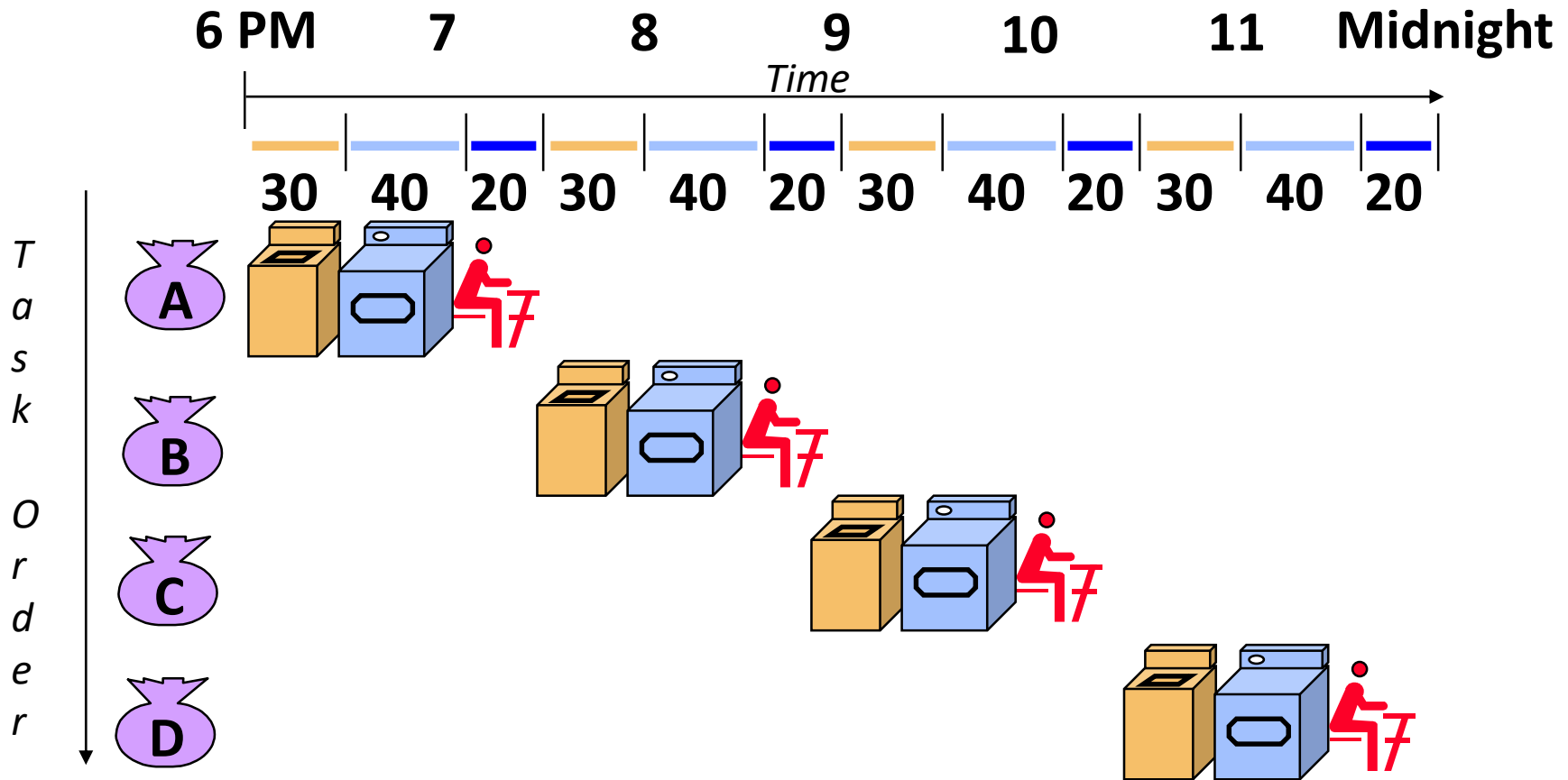
Dryer takes 40 minutes

“Folder” takes 20 minutes



Review – Pipelining

Sequential Laundry

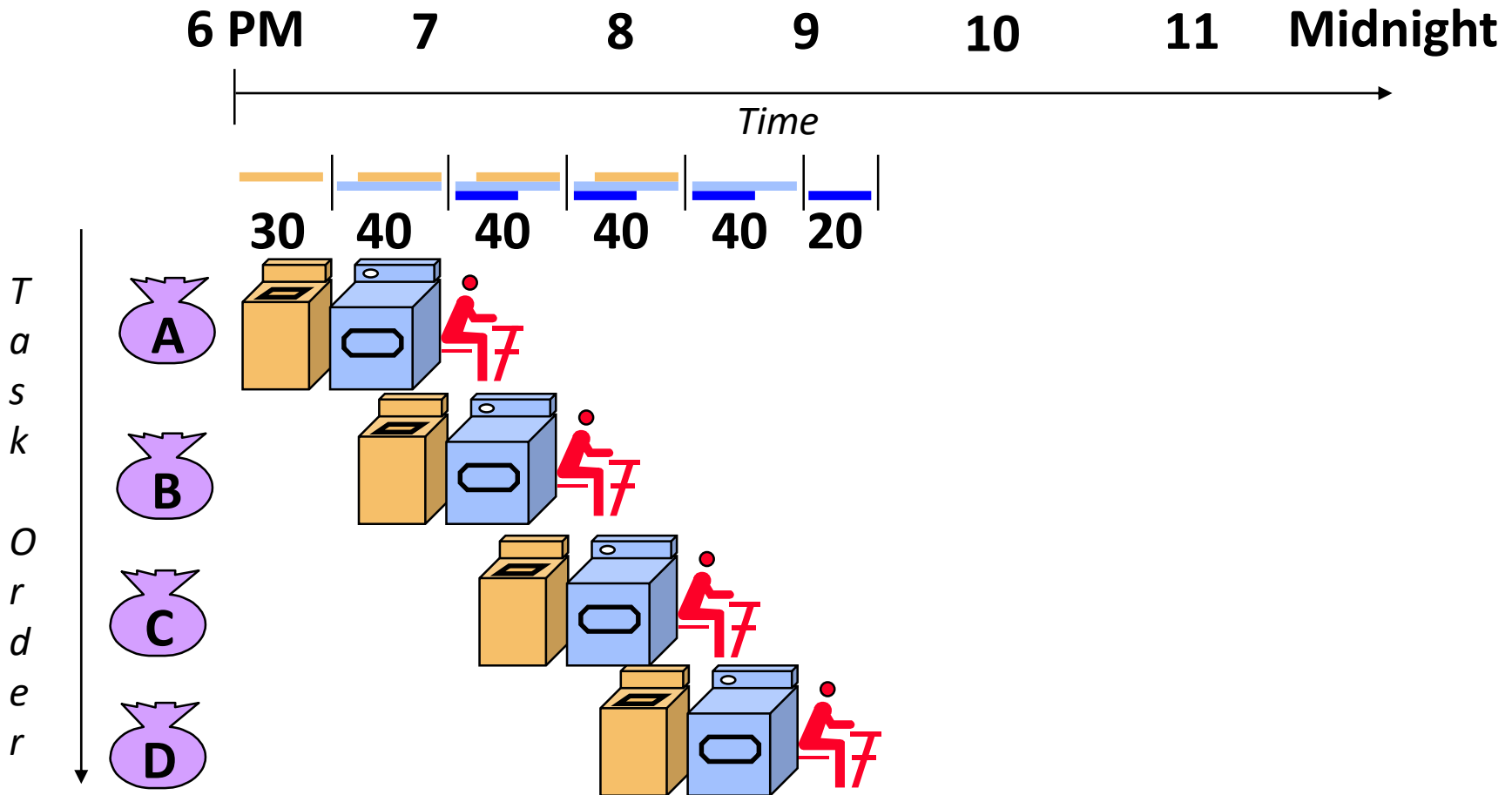


Sequential laundry takes 6 hours for 4 loads

If they learned pipelining, how long would laundry take?

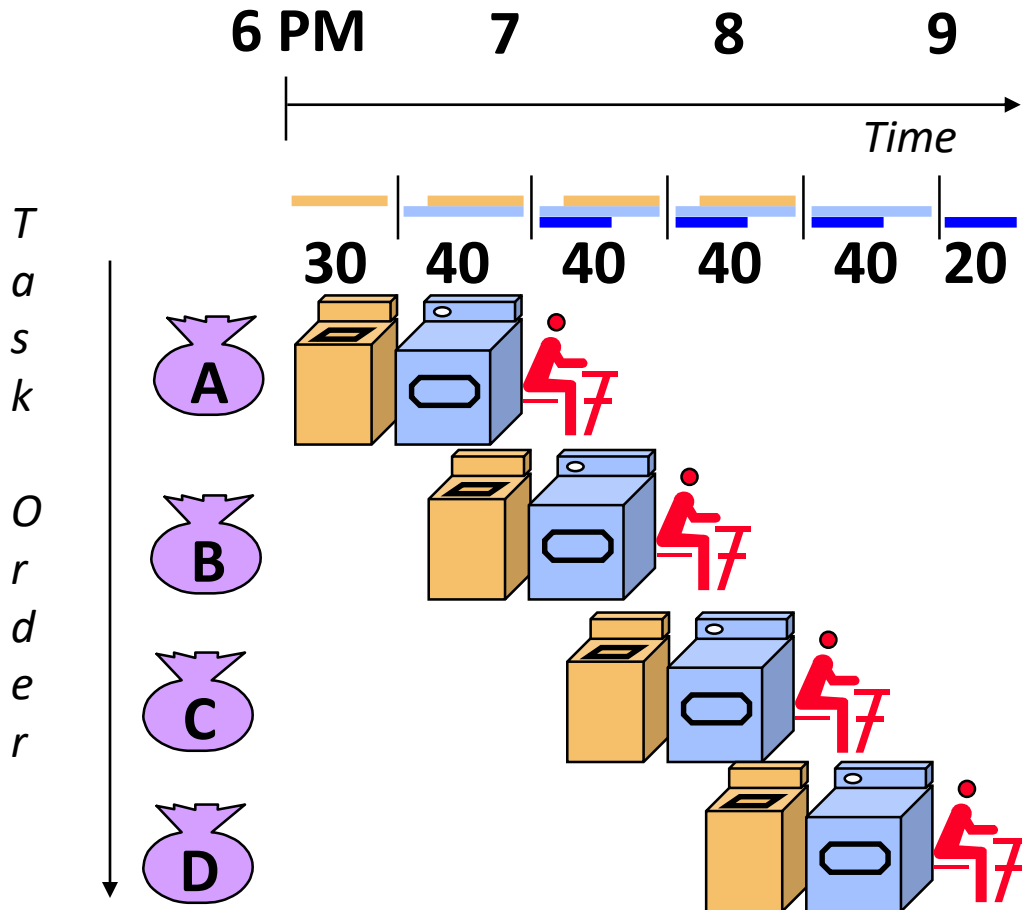
Review – Pipelining

Pipelined Laundry: Start work ASAP



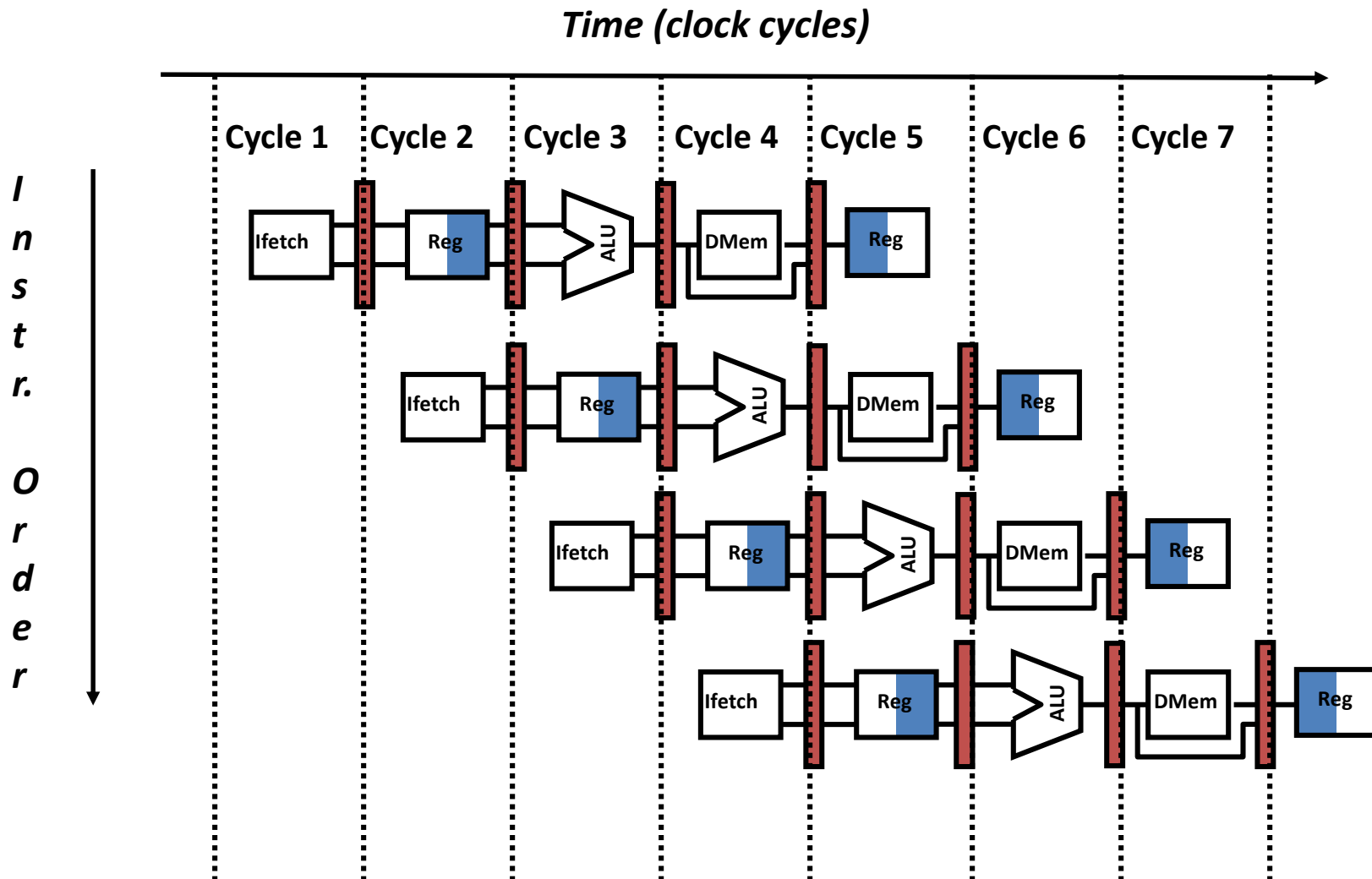
Pipelined laundry takes 3.5 hours for 4 loads

Review – Pipelining



- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- Pipeline rate limited by **slowest** pipeline stage
- **Multiple** tasks operating simultaneously
- Potential speedup = **Number pipe stages**
- Unbalanced lengths of pipe stages reduces speedup
- Time to “**fill**” pipeline and time to “**drain**” it reduces speedup

Review – Pipelining



Improving Performance – Basic Principles**

Parallelism

- Pipelining

- Multiple issue

- Multiprocessors

Locality

- Caches (review later)

Focus on common case – Amdahl's law

Amdahl's Law

(Or why the common case matters most)

Let

$$\text{Speedup} = \frac{\text{new performance}}{\text{old performance}} = \frac{\text{old latency}}{\text{new latency}}$$

Consider an enhancement x that speeds up fraction f_x of a task by S_x

$$\begin{aligned}\text{Speedup}_{\text{overall}} &= \frac{\text{old latency}}{\text{new latency}} \\ &= \frac{\{(1 - f_x) + (f_x)\} \times \text{old latency}}{(1 - f_x) \times \text{old latency} + f_x/S_x \times \text{old latency}}\end{aligned}$$

Amdahl's law gives

$$\text{Speedup}_{\text{overall}} = \frac{1}{(1 - f_x) + f_x/S_x}$$

Amdahl's Law, cont.

Example: $f_x = 95\%$ and $S_x = 1.10$

$$Speedup_{overall} = \frac{1}{(1 - 0.95) + (0.95/1.10)} = 1.094$$

Example: $f_x = 5\%$ and $S_x = 10$

$$Speedup_{overall} = \frac{1}{(1 - 0.05) + (0.05/10)} = 1.047$$

Example: $f_x = 5\%$ and $S_x = \infty$

$$Speedup_{overall} = \frac{1}{(1 - 0.05) + (0.05/\infty)} = 1.052$$

Amdahl's Law Corollary

Since $S_x \rightarrow \infty$ implies

$$Speedup_{overall} = \frac{1}{(1 - f_x) + (f_x / \infty)}$$

For all real speedups:

$$Speedup_{overall} < \frac{1}{1 - f_x}$$

Example

f_x	$1/(1-f_x)$
1%	1.01
2%	1.02
5%	1.05
10%	1.11
20%	1.25
50%	2.00

Or make the common case fast

An application?

Power

Power

Energy

Temperature

*Power***

Power = Voltage x Current

Energy = Power x Time

Temperature = Complex function of power, determined by max power drawn (averaged over short intervals)

Power and Energy

Power = Dynamic power + Static power

Energy = Power * Time

Dynamic Energy $\propto \frac{1}{2} * \text{Capacitance} * \text{Voltage}^2$

Dynamic Power $\propto \frac{1}{2} * \text{Capacitance} * \text{Voltage}^2 * \text{Frequency}$

Static power = Static current * Voltage

*Power and Energy***

Power = Dynamic power + Static power

Energy = Power * Time

Dynamic Power $\propto \frac{1}{2} * \text{Capacitance} * \text{Voltage}^2 * \text{Frequency}$

Clock gating reduces switching to reduce power

Reducing voltage reduces power

But also requires reducing frequency

Increases execution time

But reduction in power (and energy) more than increase in time

Dynamic voltage-frequency scaling (DVFS)

Static power = Static current * Voltage

Power gating reduces static (and dynamic) power

System strategy for energy: Race-to-halt

New reality: Dark Silicon

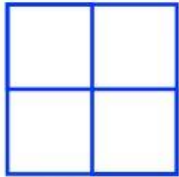
Dark Silicon

Utilization Wall: Dark Silicon's Effect on Multicore Scaling

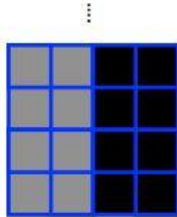
Spectrum of tradeoffs
between # of cores and
frequency

Example:
65 nm $\rightarrow$ 32 nm ($S = 2$)

4 cores @ 1.8 GHz

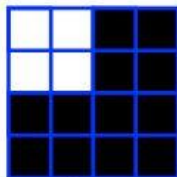


65 nm



2x4 cores @ 1.8 GHz
(8 cores dark, 8 dim)

(Industry's Choice)



4 cores @ 2x1.8 GHz
(12 cores dark)

75% dark after 2 generations;
93% dark after 4 generations

32 nm

Figure 1: Multicore scaling leads to large amounts of Dark Silicon. (From [8].)

S^2 , or $2\times$ per process generation. *This is an exponentially worsening problem that accumulates with each process generation.*

Transistor Property	Dennardian	Post-Dennardian
Δ Quantity	S^2	S^2
Δ Frequency	S	S
Δ Capacitance	$1/S$	$1/S$
ΔV_{dd}^2	$1/S^2$	1
$\Rightarrow \Delta \text{Power} = \Delta QFCV^2$	1	S^2
$\Rightarrow \Delta \text{Utilization} = 1/\text{Power}$	1	$1/S^2$

Table 1: Dennardian vs. Post-Dennardian (leakage-limited) scaling In contrast to the Classical regime proposed by Dennard [4], under the Post-Dennardian regime, the total chip utilization for a fixed power budget drops by a factor of S^2 with each process generation. The result is an exponential increase in the quantity of dark silicon for a fixed-sized chip under a fixed area budget. From [26].

Growth in Clock Rate

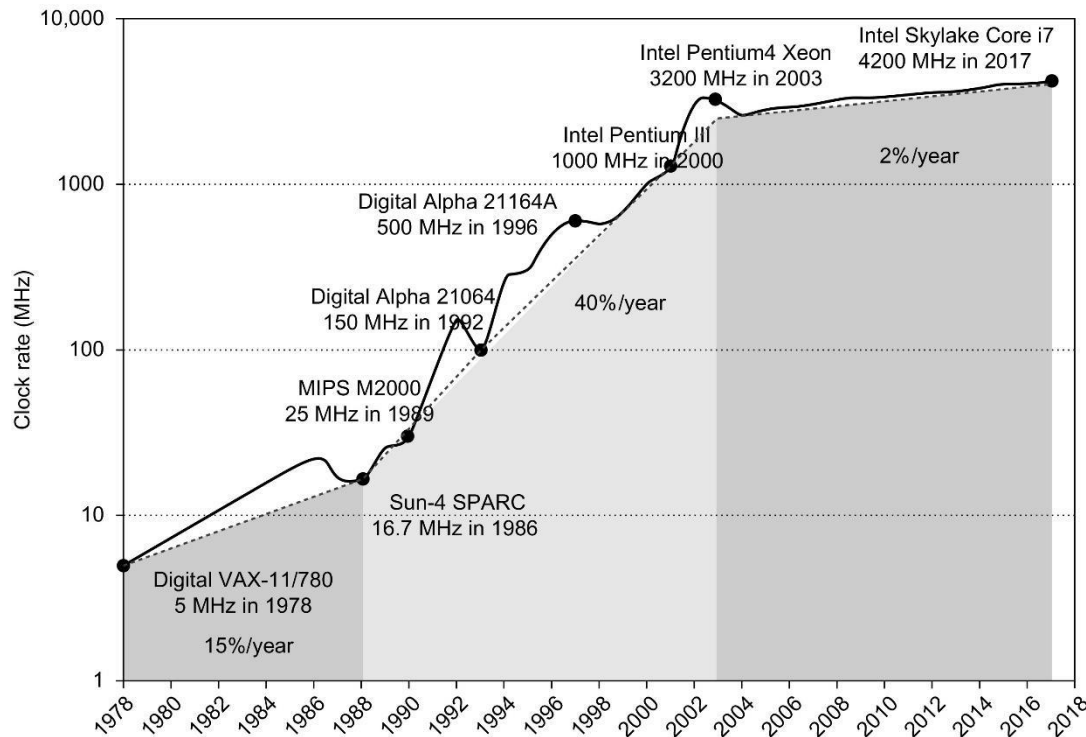


Figure 1.11 Growth in clock rate of microprocessors in Figure 1.1. Between 1978 and 1986, the clock rate improved less than 15% per year while performance improved by 22% per year. During the “renaissance period” of 52% performance improvement per year between 1986 and 2003, clock rates shot up almost 40% per year. Since then, the clock rate has been nearly flat, growing at less than 2% per year, while single processor performance improved recently at just 3.5% per year.

Cost

Cost is very important in most real designs

But usually hard to quantify for the architect

Costs change over time

Learning curve lowers manufacturing costs

Technology improvements lower costs

Focus on IC costs next

A Wafer

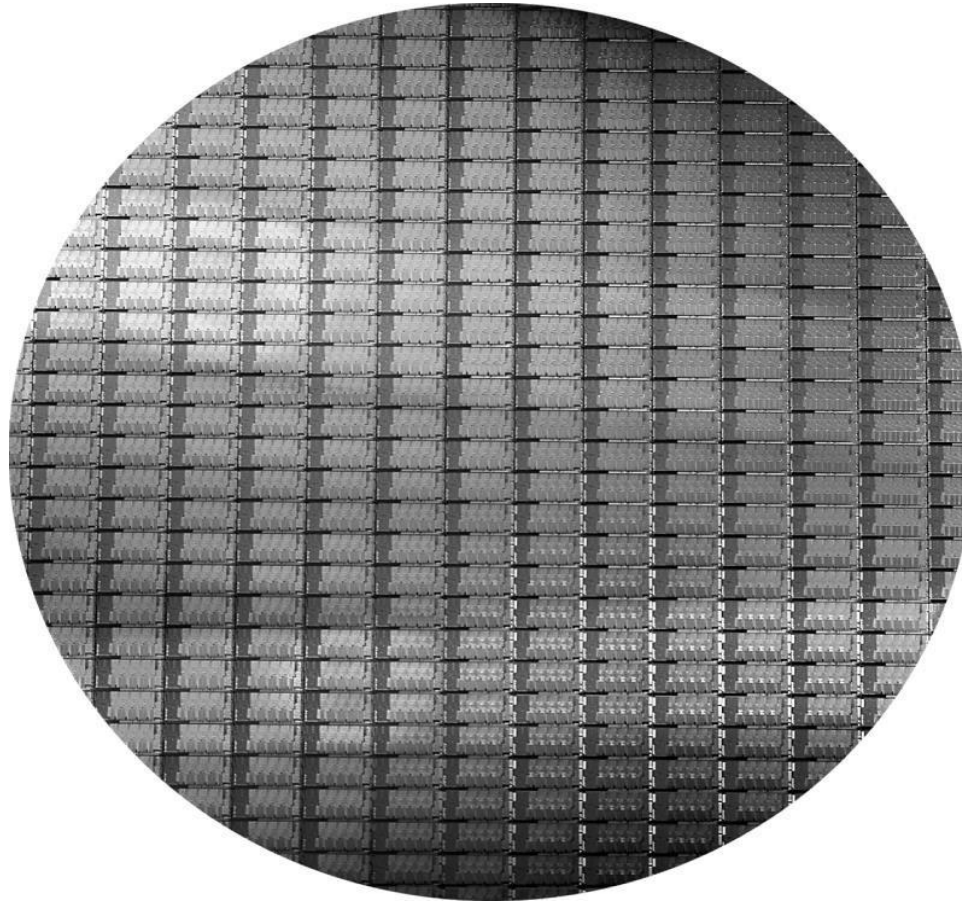


Figure 1.15 This 300 mm wafer contains 280 full Sandy Bridge dies, each 20.7 by 10.5 mm in a 32 nm process. (Sandy Bridge is Intel's successor to Nehalem used in the Core i7.) At 216 mm², the formula for dies per wafer estimates 282. (Courtesy Intel.)

A Die

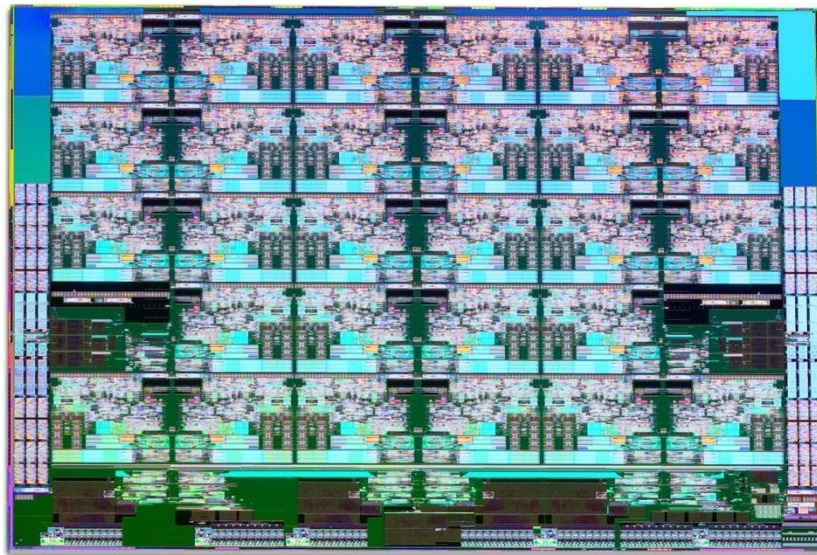


Figure 1.14 Photograph of an Intel Skylake microprocessor die, which is evaluated in Chapter 4.

A Die with Parts Labeled

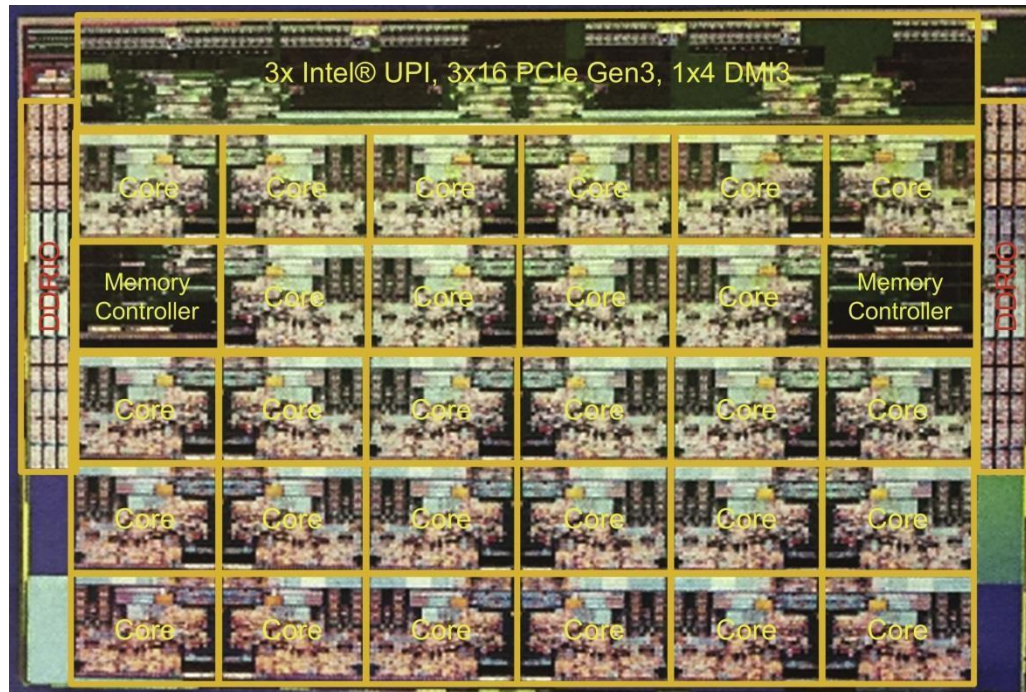


Figure 1.15 The components of the microprocessor die in Figure 1.14 are labeled with their functions.

Integrated Circuit Cost

$$\text{Cost of IC} = \frac{\text{Cost of Die} + \text{Cost of Testing} + \text{Cost of Packaging}}{\text{Final Test Yield}}$$

$$\text{Cost of Die} = \frac{\text{Cost of Wafer}}{\text{Dies per Wafer} \times \text{Die Yield}}$$

$$\text{Dies per Wafer} = \left(\frac{\pi \times (\text{Wafer Diameter}/2)^2}{\text{Die Area}} \right) -$$

(Correction factor for Edge Effects)

$$\text{Die Yield} = \text{Wafer Yield} \times \frac{1}{(1 + \text{Defects per unit area} \times \text{Die Area})^\alpha}$$

$\alpha = 10$ to 14 for 16nm in 2017

Bottom line: Cost per die grows roughly as the square of the die area

Cost different from price; cost of manufacturing different from cost of operation³²

Reliability

Many sources of unreliability

Soft errors due to radiation, hard errors due to wearout, ...

Common metrics

Mean time to failure - MTTF

For exponentially distributed time to failure

Define failures in time or FITs

FIT = failures in a billion hours

$FIT \propto 1/MTTF$

FIT of system = Sum of FITs of components

Mean time to repair - MTTR

Mean time between failures - MTBF

$MTBF = MTTF + MTTR$ *Module availability* = $\frac{MTTF}{MTTF + MTTR}$

Common solution

Reliability**

Many sources of unreliability

Soft errors due to radiation, hard errors due to wearout, ...

Common metrics

Mean time to failure – MTTF

For exponentially distributed time to failure

Define failures in time or FITs

FIT = failures in a billion hours $\text{FIT} \propto 1/\text{MTTF}$

FIT of system = Sum of FITs of components

Mean time to repair - MTTR

Mean time between failures - MTBF

MTBF = MTTF + MTTR **Module availability** = $\frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$

Common solution

Redundancy in time, space, information. But must be cheap

Many recent innovations

Acknowledgements

These slides contain material developed and copyright by:

- Sarita Adve (UIUC)
- Nathan Beckmann (CMU)
- David Patterson (UC Berkeley)
- Qianni Deng (SJTU)
- Changsheng Xie (HUST)